Agentic Systems

Institute: Indian Institute of Information Technology (IIIT) Surat
Participants: Senior Undergrads.
Lab: deepdynamicsai.github.io **Coordinator:** Dr. Pradeep Singh
Assignment 2

Problem. Build, entirely in basic Python and without using any agent framework such as LangChain, LangGraph, CrewAI, or AutoGen, an autonomous LLM-powered diagnostic agent whose task is to identify the hidden cause of failure of a simulated computing server. At the beginning of each episode, the environment secretly selects exactly one fault H from the set $\mathcal{H} = \{\texttt{network_failure}, \texttt{disk_full}, \texttt{memory_exhaustion}, \texttt{service_crash}, \texttt{configuration_error}\}$, with initial probabilities $P(H = h) = 1/5$. The agent cannot observe the hidden fault directly and must diagnose it by sequentially invoking diagnostic tools. The available tools are `ping_server`, `check_disk`, `check_memory`, `read_service_log`, and `check_config`. Each tool returns either `NORMAL` or `ABNORMAL`, and its behaviour is probabilistic. If the hidden fault is respectively network failure, disk full, memory exhaustion, service crash, or configuration error, the probabilities of receiving `ABNORMAL` from `ping_server` are $0.95, 0.05, 0.05, 0.10, 0.10$; from `check_disk` they are $0.05, 0.95, 0.10, 0.10, 0.10$; from `check_memory` they are $0.05, 0.10, 0.95, 0.10, 0.15$; from `read_service_log` they are $0.10, 0.20, 0.25, 0.90, 0.55$; and from `check_config` they are $0.05, 0.10, 0.10, 0.20, 0.90$. The corresponding tool costs are $1, 1, 1, 2, 2$, respectively. Each diagnostic tool may be used at most once in an episode, and the total diagnostic cost may not exceed $C = 5$. Conditional on the hidden fault, the outcomes of different diagnostic tests may be treated as independent.

The agent must explicitly maintain a memory $M_t = \{(a_1, o_1), \ldots, (a_t, o_t)\}$, where a_t is the diagnostic action selected at step t and $o_t \in \{\texttt{NORMAL}, \texttt{ABNORMAL}\}$ is the corresponding observation. It must also maintain a belief state b_t containing the posterior probability of every possible fault. After observing o_t following action a_t, update each hypothesis using Bayes' rule $P(H = h \mid o_t, a_t, M_{t-1}) = P(o_t \mid H = h, a_t)P(H = h \mid M_{t-1}) / \sum_{h'} P(o_t \mid H = h', a_t)P(H = h' \mid M_{t-1})$. The uncertainty in the current belief should be measured using the entropy $\mathcal{H}(b_t) = -\sum_h P(H = h \mid M_t) \log_2 P(H = h \mid M_t)$. All Bayesian calculations, memory management, tool execution, budget accounting, validation, and stopping logic must be implemented explicitly in Python.

The decision-making policy of the agent must use a small openly available instruction-following LLM, preferably `Qwen/Qwen3-4B-Instruct-2507`, accessed through an API such as the Hugging Face Inference API. At each step, the Python program should construct a prompt containing only information legitimately available to the agent: the current posterior probabilities, previous diagnostic actions and observations, unused tools, their costs, the remaining budget, and the permissible actions. The LLM must choose either one unused diagnostic tool or the action `STOP`. Its response must follow a fixed machine-readable format such as JSON containing the selected action and a brief reason. The LLM must not execute tools, generate diagnostic outcomes, update posterior probabilities, modify memory, alter the budget, or directly control the environment. Instead, it should act only as the policy $\pi_{\text{LLM}}(b_t, M_t, C_t)$ proposing the next action, while the surrounding Python controller validates and executes that action. Any malformed response, unavailable tool, repeated tool call, or action whose cost exceeds the remaining budget must be rejected by the controller.

In addition to the LLM-based policy, implement a purely mathematical policy based on expected information gain. For every unused action a, compute $IG(a) = \mathcal{H}(b_t) - \sum_o P(o \mid a, M_t)\mathcal{H}(b_{t+1}^{(o)})$, and use $IG(a)/c(a)$ to quantify the expected reduction in uncertainty per unit cost. Implement a baseline agent that always selects the affordable unused diagnostic tool having the largest value of $IG(a)/c(a)$. Also implement a random baseline that selects uniformly from the currently affordable unused tools. The diagnostic agent may terminate when $\max_h P(H = h \mid M_t) \geq 0.90$, when no further affordable diagnostic tool remains, or when the permitted budget has been exhausted. Its final diagnosis should be $\hat{h} = \arg\max_h P(H = h \mid M_t)$. If the LLM proposes `STOP` before the confidence threshold has been reached while another affordable diagnostic action remains, the deterministic controller should reject the request and require another valid action.

Evaluate the random policy, the maximum-information-gain-per-cost policy, and the Qwen-powered policy over at least 200 independently generated diagnostic episodes using fixed random seeds for reproducibility. For each policy, report diagnostic accuracy, average diagnostic cost, average number of diagnostic tool calls, final posterior confidence, and, for the LLM agent, the average number of LLM calls and the number of invalid actions proposed by the model. Additionally compute the episode loss $J = 100\,\mathbf{1}\{\hat{h} \neq h^*\} + \sum_t c(a_t) + 0.1N_{\text{LLM}}$, where h^* is the true hidden fault and N_{LLM} is the number of LLM calls. The objective of the exercise is to implement and understand the complete agent loop belief state → LLM decision → action validation → tool execution → observation → Bayesian update → memory update → next decision or stop, and to compare an LLM-driven decision policy with transparent mathematical and random baselines.